

LocalLLM-DataForge: Un Pipeline Modular para Conjuntos de Datos Sintéticos de Descomposición de Historias de Usuario con Validación tipo LLM-as-a-Judge

Angel Luis Valdés Sánchez¹,
Carlos Alberto Fernández y Fernández^{2*},
Verónica Rodríguez López²

¹División de Estudios de Posgrado, Universidad Tecnológica de la Mixteca, Av. Dr. Modesto Seara Vázquez No. 1, Huajuapán de León, 69004, Oaxaca, México.

^{2*}Instituto de Computación, Universidad Tecnológica de la Mixteca, Av. Dr. Modesto Seara Vázquez No. 1, Huajuapán de León, 69004, Oaxaca, México.

*Corresponding author(s). E-mail(s): caff@gs.utm.mx;
Contributing authors: alvaldes.dev@gmail.com; veromix@gs.utm.mx;

Abstract

Descomponer historias de usuario en tareas de desarrollo concretas es una práctica habitual en los proyectos ágiles, aunque sigue siendo una actividad que depende en gran medida del juicio humano. Si bien automatizar este paso mediante técnicas de aprendizaje automático resulta atractivo, en la práctica suele enfrentarse a dos problemas recurrentes: la escasez de datos anotados y el costo asociado al uso de LLMs basados en la nube. Este trabajo presenta LocalLLM-DataForge, un framework modular basado en DataFrames que permite la generación de conjuntos de datos sintéticos utilizando LLMs ejecutados de forma local. El sistema se apoya en Ollama para la ejecución de los modelos, incorpora una capa de caché inteligente para evitar cálculos redundantes y aplica verificaciones automáticas de calidad inspiradas en enfoques de LLM como juez. Se exploran dos estrategias de evaluación: una evalúa un único generador a partir de cinco criterios explícitos, mientras que la otra compara las salidas de dos generadores y selecciona la descomposición más sólida. Los experimentos realizados sobre el conjunto de datos Salony, utilizando modelos como Llama 3.1

y Mistral, sugieren que el uso de caché reduce de forma significativa los ciclos de experimentación y que las descomposiciones generadas mantienen coherencia lógica y una alineación razonable con el criterio de expertos. En conjunto, el framework ofrece una aproximación reproducible para la ingeniería de requisitos sin depender de servicios en la nube.

Keywords: Descomposición de Historias de Usuario, Generación de Datos Sintéticos, LLMs Locales, Ollama, LLM como Juez, Ingeniería de Requisitos Ágil

1 Introducción

Las historias de usuario son uno de los artefactos de requisitos más utilizados en el desarrollo de software ágil. Los equipos suelen apoyarse en ellas para capturar necesidades funcionales desde la perspectiva del usuario final de forma concisa y accesible [1, 2]. En la práctica, su función va más allá de la comunicación: las historias de usuario se emplean de manera habitual para apoyar la planificación, la estimación de esfuerzo y las actividades cotidianas de desarrollo. En particular, la descomposición de las historias en tareas de desarrollo concretas resulta crítica durante la refinación del backlog y la planificación de sprints [3, 4]. Aun así, esta actividad sigue dependiendo en gran medida del trabajo manual y de la experiencia acumulada del equipo.

Cuando la descomposición de tareas se realiza de forma manual, tienden a aparecer una serie de problemas recurrentes. Distintos miembros del equipo pueden interpretar una misma historia de usuario de maneras ligeramente diferentes, lo que da lugar a niveles inconsistentes de granularidad y a supuestos implícitos que nunca llegan a documentarse por completo. Algunos detalles relevantes pueden pasarse por alto (e.g., restricciones funcionales implícitas), mientras que otros aspectos se descomponen en exceso. Con el tiempo, estas inconsistencias contribuyen a la creación de backlogs poco estructurados, lo que dificulta la estimación y la asignación de tareas [5]. El problema suele intensificarse en equipos grandes o distribuidos, donde la ausencia de prácticas compartidas de descomposición debilita la trazabilidad y complica la automatización sistemática de los flujos de trabajo en ingeniería de requisitos [6].

Los avances recientes en Procesamiento del Lenguaje Natural (NLP, por sus siglas en inglés), junto con la rápida evolución de los Modelos de Lenguaje Grande (LLMs, por sus siglas en inglés), han abierto nuevas posibilidades para automatizar tareas que tradicionalmente quedaban en manos de ingenieros de software [7, 8]. Trabajos previos han explorado el uso de LLM para la clasificación de requisitos, la detección de ambigüedades, la generación de casos de prueba y la derivación de tareas de desarrollo a partir de descripciones en lenguaje natural [9, 10]. Sin embargo, la mayoría de las soluciones existentes dependen de modelos propietarios alojados en la nube. Esta dependencia introduce preocupaciones prácticas relacionadas con el costo, la confidencialidad de los datos, la reproducibilidad y la disponibilidad a largo plazo, tanto en contextos de investigación como industriales.

Una limitación compartida por muchos enfoques basados en LLM es la falta de conjuntos de datos adecuados para entrenamiento y evaluación. Los datasets públicos

centrados específicamente en la descomposición de historias de usuario siguen siendo escasos, en gran medida debido al elevado costo de la anotación por expertos y a la naturaleza sensible de los requisitos del mundo real [11]. La generación de datos sintéticos mediante LLM se ha propuesto como una solución parcial. No obstante, muchos de los métodos existentes ofrecen poca visibilidad sobre cómo se validan las salidas generadas. Como consecuencia, evaluar la corrección semántica, la completitud o la coherencia lógica resulta complicado [12]. Además, una generación poco controlada incrementa el riesgo de alucinaciones, lo que puede reducir la confianza en los conjuntos de datos resultantes.

Este trabajo presenta LocalLLM-DataForge, un framework modular diseñado para generar conjuntos de datos sintéticos orientados a la descomposición de historias de usuario utilizando LLMs ejecutados de forma local. El framework se apoya en DataFrames para estructurar y gestionar los resultados intermedios del proceso de generación. Asimismo, se integra con Ollama para ejecutar modelos de lenguaje de código abierto sin depender de servicios en la nube. Uno de sus componentes centrales es un mecanismo de caché inteligente, que evita cálculos repetidos durante experimentos iterativos y contribuye a reducir los tiempos de ejecución al explorar distintas configuraciones de generación.

El control de calidad se aborda mediante mecanismos de validación automática. LocalLLM-DataForge aplica estrategias de LLM como juez para evaluar las descomposiciones de tareas generadas. Se consideran dos enfoques de evaluación, el primero se basa en un único generador y evalúa sus salidas a partir de criterios de calidad explícitos (i.e., claridad, atomicidad, coherencia lógica, relevancia y completitud) [13]. El segundo compara las salidas producidas por múltiples generadores y selecciona la descomposición más adecuada. Esta estrategia comparativa aprovecha principios de consenso que han demostrado mejorar la fiabilidad y reducir errores semánticos en las salidas de los modelos [14, 15].

Este trabajo realiza tres contribuciones principales. En primer lugar, presenta un framework completamente local, extensible y reproducible para la generación de conjuntos de datos sintéticos a partir de historias de usuario, abordando de forma explícita restricciones de costo y privacidad. En segundo lugar, define y operacionaliza estrategias automáticas de validación basadas en LLM como juez, adaptadas a artefactos de ingeniería de requisitos. Por último, reporta resultados empíricos sobre el conjunto de datos Salony utilizando varios LLM de código abierto, destacando tanto las mejoras de eficiencia derivadas del uso de caché como las mejoras de calidad obtenidas mediante la selección comparativa de modelos.

El resto del artículo se organiza de la siguiente manera. La Sección 2 revisa el trabajo relacionado sobre automatización basada en LLM en ingeniería de requisitos y generación de datos sintéticos. La Sección 3 describe la arquitectura y los componentes principales de LocalLLM-DataForge. La Sección 4 presenta la configuración experimental y la metodología de evaluación. La Sección 5 discute los resultados obtenidos, mientras que la Sección 6 expone las conclusiones y las líneas de trabajo futuro.

2 Trabajo Relacionado

3 Framework LocalLLM-DataForge

4 Métodos

La herramienta LocalLLM-DataForge se empleó para diseñar y ejecutar un estudio exploratorio de carácter experimental orientado a evaluar la viabilidad de la descomposición sintética automática de historias de usuario mediante modelos de lenguaje ejecutados localmente. El entorno experimental consistió en un equipo Apple con chip M2 (8 núcleos de CPU), 16 GB de memoria RAM y ejecutando Python v3.13.7. Se utilizó Ollama v0.16.1 como plataforma de orquestación para la ejecución local de los modelos LLMs. Todo el procesamiento relacionado con la generación de datos sintéticos y su posterior evaluación se realizó íntegramente en este hardware local, sin depender de servicios en la nube, lo que permitió garantizar reproducibilidad, control de recursos computacionales y confidencialidad de los datos procesados.

El conjunto de datos base empleado fue Salony¹, el cual contiene originalmente 1,999 historias de usuario. Con el fin de asegurar consistencia estructural y evitar ambigüedades derivadas de formatos heterogéneos, se realizó un proceso de limpieza previa en el cual únicamente se conservaron aquellas historias que cumplían estrictamente con la estructura canónica “*As a(n) [role], I want [feature] so that [benefit]*”. Después de este filtrado, el conjunto final quedó conformado por 1,139 historias de usuario válidas. Estas historias constituyeron el DataFrame inicial del pipeline y fueron utilizadas exclusivamente con fines de generación sintética; no se realizó partición en conjuntos de entrenamiento y prueba, dado que el objetivo del estudio no fue entrenar modelos predictivos sino evaluar la capacidad generativa y la viabilidad metodológica del enfoque propuesto.

Una vez definido el conjunto de datos experimental, se procedió al diseño de la arquitectura de procesamiento. Esta se implementó mediante la clase `DataForgePipeline`, que permite encadenar pasos especializados bajo un diseño modular basado en DataFrames. En una primera etapa, el componente `LoadDataFrame` se encargó de cargar el conjunto de datos previamente depurado dentro de la tubería de procesamiento. Posteriormente, se configuraron dos instancias independientes del componente `OllamaLLMStep`, correspondientes a dos generadores diferenciados. El Generador A utilizó el modelo `llama3.1:8b`, con un tamaño de lote (`batch_size`) de 2, temperatura de 0.3 para favorecer un comportamiento más determinista y un máximo de 1,000 tokens de salida (`num_predict`). El Generador B empleó el modelo `qwen3:8b`, también con `batch_size` de 2 y límite de 1,000 tokens, pero con una temperatura de 0.7 para promover una mayor creatividad en la generación de tareas. La selección de estas configuraciones respondió al interés de contrastar un modelo con sesgo determinista frente a otro con mayor variabilidad generativa, permitiendo observar diferencias cualitativas en los patrones de descomposición producidos.

¹Salony: User Story Dataset disponible en Hugging Face, https://huggingface.co/datasets/salony/User_story.

Cada generador produjo una lista independiente de tareas para cada historia de usuario, almacenándose los resultados en columnas separadas del DataFrame. Dado que cada historia produce dos descomposiciones independientes se requirió un mecanismo sistemático de evaluación comparativa. Esta etapa se implementó mediante el componente `ComparisonJudgeStep`, el cual utiliza el patrón LLM-as-a-Judge para contrastar ambas salidas. Este enfoque consiste en emplear un modelo de lenguaje no como generador, sino como evaluador estructurado de artefactos producidos por otros modelos. Este paradigma ha sido adoptado recientemente en investigaciones experimentales donde las métricas automáticas tradicionales resultan insuficientes para capturar calidad semántica, coherencia estructural o adecuación contextual. En este estudio, el modelo evaluador (llama3.1:8b) fue configurado con una temperatura reducida (0.2) y `batchsize` de 1, con el objetivo de maximizar la consistencia entre los juicios y minimizar la variabilidad estocástica. Su función no fue reemplazar la evaluación humana, sino proporcionar un mecanismo sistemático, reproducible y escalable para analizar la calidad de las descomposiciones generadas bajo criterios explícitamente definidos.

El diseño experimental puede caracterizarse como un estudio exploratorio de viabilidad en el que la variable independiente fue la configuración del generador (modelo y temperatura), mientras que la variable dependiente principal fue la puntuación total de calidad asignada por el modelo evaluador. Se realizaron múltiples ejecuciones experimentales variando ajustes de prompts y parámetros con el objetivo de observar consistencia en los resultados y analizar el comportamiento de los modelos bajo distintas condiciones de configuración.

Las llamadas a los modelos se procesaron mediante un esquema de procesamiento por lotes configurable con el fin de optimizar el uso de memoria y evitar desbordamientos del contexto máximo permitido por los modelos locales. El tamaño de lote fue determinado considerando las limitaciones de memoria del equipo y la longitud potencial de las salidas generadas. Asimismo, se habilitó el sistema de caché interno del pipeline mediante un parámetro opcional, lo que permitió evitar recomputaciones cuando las entradas y configuraciones permanecían constantes. Esta estrategia redujo los tiempos de experimentación y favoreció la replicabilidad exacta de los resultados en ejecuciones sucesivas. El pipeline completo fue definido en scripts reproducibles dentro del repositorio del proyecto, tales como `examples/salonypipeline.py` y `examples/salonydualgeneratorpipeline.py`, que pueden ser ejecutados directamente desde la línea de comandos con Python.

Una vez descrita la configuración técnica del pipeline y sus mecanismos de ejecución, resulta fundamental detallar el esquema de evaluación adoptado en el estudio. Para cada historia de usuario se consideraron cinco criterios de evaluación: coherencia, completitud, viabilidad, formato y granularidad. Cada criterio fue puntuado en una escala de 0 a 10, obteniéndose una puntuación total máxima de 50 puntos. La selección de estos criterios responde a la necesidad de descomponer el concepto abstracto de “calidad” en dimensiones observables y evaluables, siguiendo principios comúnmente aceptados en evaluación de artefactos de ingeniería de requisitos.

La coherencia evaluó la correspondencia semántica directa entre las tareas generadas y el contenido explícito de la historia de usuario, verificando que no existieran

tareas fuera de alcance ni interpretaciones erróneas del objetivo funcional. La completitud midió el grado en que la descomposición cubre todos los aspectos funcionales implícitos y explícitos de la historia, identificando posibles omisiones críticas que comprometerían la implementación. La viabilidad examinó si las tareas propuestas son técnicamente realizables dentro de un entorno de desarrollo estándar, descartando pasos imposibles, redundantes o lógicamente inconsistentes. El criterio de formato analizó la claridad estructural de la respuesta, verificando que cada tarea incluyera los elementos requeridos (resumen y descripción) y que la organización general facilitara su interpretación y eventual incorporación en herramientas de gestión ágil. Finalmente, la granularidad evaluó si el nivel de descomposición era apropiado: tareas excesivamente amplias dificultan la planificación incremental, mientras que tareas demasiado atómicas generan fragmentación innecesaria y sobrecarga administrativa.

La evaluación multidimensional permitió evitar juicios binarios simplificados y ofreció una representación más rica del desempeño del modelo. Esta estrategia es especialmente relevante en contextos donde la calidad no puede reducirse a coincidencias léxicas, sino que depende de adecuación conceptual y estructural.

Por otra parte, el diseño de los prompts constituyó un componente metodológico clave dentro del experimento. El prompt de generación fue estructurado bajo un esquema tipo instrucción-entrada-respuesta, donde se especifica explícitamente que la historia debe descomponerse en tareas únicas, accionables y no superpuestas, imponiendo además un formato rígido basado en numeración secuencial y campos obligatorios de resumen y descripción. Esta restricción estructural tuvo como finalidad reducir variabilidad en la salida y facilitar el procesamiento automatizado posterior dentro del DataFrame. Por su parte, el prompt del modelo evaluador fue diseñado para operar como un mecanismo de validación multidimensional, definiendo explícitamente los cinco criterios de evaluación y estableciendo reglas críticas de rechazo automático en caso de incumplimiento del umbral mínimo o de presencia de errores estructurales graves. Se exigió que la respuesta del juez fuera emitida exclusivamente en formato JSON, sin texto adicional, con el propósito de garantizar la correcta extracción programática de las métricas generadas.

Se estableció un umbral de aprobación de 35 puntos (70% del máximo posible), criterio que garantiza que una descomposición cumpla con una proporción sustancial de los atributos de calidad definidos. El modelo evaluador generó automáticamente las puntuaciones parciales y la puntuación total para cada instancia, registrando además un indicador booleano de aprobación. Cuando la puntuación resultaba inferior al umbral establecido, el sistema almacenaba adicionalmente una lista de problemas identificados y recomendaciones de mejora. Paralelamente, evaluadores humanos expertos revisaron una muestra representativa de las descomposiciones con el objetivo de contrastar la consistencia de los juicios automáticos y determinar la confiabilidad del enfoque LLM-as-a-Judge. En este esquema híbrido, la evaluación humana se consideró el criterio definitivo de validación, mientras que el modelo evaluador actuó como mecanismo de apoyo estructurado y escalable.

Los resultados de evaluación fueron almacenados en columnas específicas del DataFrame, tales como `validacion_coherencia`, `validacion_completitud`, `validacion_viabilidad`, `validacion_formato`, `validacion_granularidad`,

`validacion_total` y `validacion_aprobado`, lo cual permitió realizar análisis posteriores de manera sistemática.

En conjunto, la metodología fue diseñada para ser modular, reproducible y ejecutable en entornos locales con recursos limitados. La combinación de arquitectura basada en pasos encadenados, procesamiento por lotes, configuraciones explícitas de hiperparámetros y sistema de caché asegura trazabilidad completa del proceso experimental y permite replicar los resultados bajo las mismas condiciones de configuración y versiones de software, consolidando así la validez técnica del estudio de viabilidad presentado.

5 Discusión

6 Conclusión

Agradecimientos.

Declaraciones

- Financiamiento: No aplica
- Conflicto de intereses: Los autores declaran no tener conflictos de interés.
- Aprobación ética y consentimiento de participación: No aplica
- Consentimiento para publicación: No aplica
- Disponibilidad de datos: Los conjuntos de datos y el código del pipeline están disponibles en el repositorio del proyecto.
- Disponibilidad de código: El código fuente de LocalLLM-DataForge está disponible públicamente.
- Contribución de autores: Todos los autores contribuyeron al diseño y desarrollo del framework. A.L.V.S. lideró la implementación y los experimentos. C.A.F.F. y V.R.L. supervisaron la investigación y contribuyeron a la metodología.

References

- [1] Cohn, M.: User Stories Applied: For Agile Software Development. Addison-Wesley, ??? (2004)
- [2] Beck, K.: Extreme Programming Explained: Embrace Change. Addison-Wesley, ??? (2001)
- [3] Rubin, K.S.: Essential Scrum: A Practical Guide to the Most Popular Agile Process. Addison-Wesley, ??? (2012)
- [4] Schwaber, K., Sutherland, J.: The Scrum Guide. <https://scrumguides.org>. Accessed: 2026-03-02 (2020)

- [5] Lucassen, G., Dalpiaz, F., Werf, J.M.E., Brinkkemper, S.: Improving agile requirements: the quality user story framework and tool. *Requirements Engineering* **21**(3), 383–403 (2016) <https://doi.org/10.1007/s00766-016-0250-x>
- [6] Gorschek, T., Garre, P., Larsson, S., Wohlin, C.: A model for technology transfer in practice. *IEEE Software* **23**(6), 88–95 (2006) <https://doi.org/10.1109/MS.2006.147>
- [7] Chen, M., et al.: Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021)
- [8] White, J., Fu, Q., Hays, S., Sandborn, M., Oleynik, V., et al.: A prompt pattern catalog to enhance prompt engineering with large language models. *arXiv preprint arXiv:2302.11382* (2023)
- [9] Rahimi, M., Gervasi, V.: Natural language processing for requirements engineering: A systematic mapping study. *IEEE Access* **11**, 25130–25152 (2023) <https://doi.org/10.1109/ACCESS.2023.3251234>
- [10] Sebban, M., et al.: Large language models for software requirements engineering: Opportunities and challenges. *Empirical Software Engineering* (2024)
- [11] Fucci, D., Palomares, C., Franch, X.: Data-driven requirements engineering: Models and methods. *Requirements Engineering* **25**, 1–24 (2020) <https://doi.org/10.1007/s00766-019-00315-3>
- [12] Liu, Y., et al.: Synthetic data generation with large language models: Challenges and opportunities. *arXiv preprint arXiv:2305.12867* (2023)
- [13] Chiang, D., Lee, J.-U.: Large language models are reasoning teachers. *arXiv preprint arXiv:2305.08328* (2023)
- [14] Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2023)
- [15] Li, Y., et al.: Consensus-based evaluation of large language model outputs. *arXiv preprint arXiv:2401.01335* (2024)